

Formalizing Analytic Number Theory in Lean 4: Foundations, Practices, and Lessons from the Cathedral Project

Jason Robert Gochanour [ORCID](#)

June 1, 2026

MSC 2020: 03B35 (primary), 11M26, 68V15 (secondary).

Keywords: formal verification, Lean 4, Mathlib, analytic number theory, Riemann Hypothesis, proof engineering.

Abstract

We describe the practical techniques and logical foundations for formalizing analytic number theory in Lean 4 with Mathlib, distilled from the Cathedral project: a $\sim 120,000$ -line formalization of the Nyman–Beurling criterion for the Riemann Hypothesis.

The first half covers foundations: the type-theoretic setting (CIC with extensions), the trust chain from silicon to theorem, the precise logical strength of the formalization (provable in ZFC), and a constructivity analysis showing the converse direction is partially constructive while the forward direction is inherently classical.

The second half covers practice: Mathlib API patterns that worked well (interval integrals, spectral theory), patterns requiring workarounds (infinite sums, measure theory), 5 original Mathlib gaps (3 graduated by the project, 2 remaining), 12 essential lemmas, project structure advice, and common tactic patterns. We document 4 deprecated-API traps (including the `push_neg` $\rightarrow$ `push_not` migration) and provide a verification checklist for large-scale proof engineering.

Contents

1	Introduction	2
2	The Type-Theoretic Foundation	2
2.1	CIC with Extensions	2
2.2	The Axioms of the Crown	3
2.3	Constructivity Analysis	3
2.4	The de Bruijn Criterion	3
3	The Axiom-Theorem Gradient	3
4	Mathlib APIs That Worked Well	4
4.1	Interval Integrals	4
4.2	Matrix API	4
4.3	Spectral Theory	4
4.4	Convergence	4
5	Mathlib APIs That Required Workarounds	4
5.1	Infinite Series	4
5.2	Fractional Part	4
5.3	Measure Theory Coercions	4

6	Mathlib Gaps: Two Remaining, Three Graduated	5
7	Twelve Essential Mathlib Lemmas	5
8	Project Structure	5
8.1	The Layered Module Pattern	5
8.2	The Axiom Registry Pattern	6
8.3	Sorry Hygiene	6
9	Tactic Patterns	6
10	Deprecated API Traps	6
11	Verification Checklist	6
12	Conclusion	7

1 Introduction

Every machine-verified proof rests on a stack of trust. This paper examines that stack for the Cathedral, a Lean 4 formalization establishing $\text{RH} \iff d_N^2 \rightarrow 0$, from two perspectives: the *foundational* (what are we trusting?) and the *practical* (how do we build it?).

Layer	What Must Be Trusted
5	Mathematical axioms (2 substantive on crown, ~ 92 active total)
4	Mathlib library ($\sim 1.4\text{M}$ LOC)
3	Lean kernel axioms (3: <code>propext</code> , <code>choice</code> , <code>Quot.sound</code>)
2	Lean type-checker ($\sim 5,000$ LOC of C++ kernel)
1	Compiler, OS, hardware

The Cathedral project encountered the challenge of using complex analysis, real analysis, combinatorics, and linear algebra *together* in a single proof, with consistent type coercions and compatible assumptions. Over ~ 68 days it produced 390+ active Lean files (with 114 archived). This paper distills the lessons.

2 The Type-Theoretic Foundation

2.1 CIC with Extensions

Lean 4 is based on the Calculus of Inductive Constructions (CIC) with dependent function types, inductive types, a universe hierarchy, proof irrelevance, and large elimination. Three kernel axioms extend the core:

1. `propext`: $(P \iff Q) \rightarrow P = Q$. Essential for set-theoretic reasoning.
2. `Classical.choice`: For any nonempty type A , there exists $a : A$. Implies excluded middle.
3. `Quot.sound`: If $a \sim b$ then $[a] = [b]$ in the quotient type.

The system `CIC + propext + choice + Quot.sound`, with its infinite universe hierarchy, is equiconsistent with ZFC plus countably many inaccessible cardinals [4].

2.2 The Axioms of the Crown

The crown theorem (`baez_duarte_forward`) reports 7 axioms via `#print axioms`: 3 kernel + 2 substantive + 2 PNT bureaucracy.

#	Axiom	Origin	Content
1	<code>propext</code>	Lean kernel	—
2	<code>Classical.choice</code>	Lean kernel	—
3	<code>Quot.sound</code>	Lean kernel	—
4	<code>gram_form_upper_bound</code>	Cathedral	$v^T G v \leq 1 + K/\ln N$ ($\equiv$ RH)
5	<code>mertens_34_unconditional</code>	PNT	$ M(x) \leq Cx^{3/4}$ (unconditional)
6	<code>frac_error_isLittleO</code>	PNT	Fractional-part error
7	<code>pnt_mu_log_sq_div_k</code>	PNT	$\sum \mu(n)(\ln n)^2/n$

The two *substantive* mathematical axioms are: `gram_form_upper_bound` (the Gram quadratic form bound, formally equivalent to RH) and `mertens_34_unconditional` (the classical Mertens bound $|M(x)| \leq Cx^{3/4}$, an unconditional consequence of PNT, provable in ZFC). The former sole axiom `discrete_riemann_hypothesis` ($v^T C v \leq C/\ln N$) was **graduated to a theorem** on June 1, 2026, derived via variance decomposition. Ten PNT bureaucracy axioms were graduated using the PrimeNumberTheoremAnd bridge (May 31, 2026). The 2 remaining PNT bureaucracy axioms (axioms 6–7) are unconditional consequences of the Prime Number Theorem; they are imported as axioms because Mathlib does not yet export the specific asymptotic forms needed.

2.3 Constructivity Analysis

The converse ($d_N^2 \rightarrow 0 \implies \text{RH}$) is partially constructive: the bound $d_N^2 \geq \delta > 0$ is computed explicitly given ρ ; only the existential over ρ in $\neg\text{RH}$ is non-constructive. The forward direction is inherently classical, requiring the Montgomery–Vaughan MVT and Perron’s formula.

2.4 The de Bruijn Criterion

The de Bruijn criterion states that a formal verification system should have a small, isolated trusted computing base (the kernel) so that proof correctness does not depend on the massive complexity of the tactics or library [5].

Component	Size (LOC)
Lean kernel (type-checker)	~5,000
Lean elaborator + tactics	~100,000
Mathlib library	~1,400,000
Cathedral	~120,000

Only the kernel (5,000 LOC) is in the trusted base.

3 The Axiom-Theorem Gradient

The Cathedral introduces a foundational concept: the *axiom-theorem gradient*.

1. **Kernel axioms**: Trusted by design. (3)
2. **Crown axioms (substantive)**: On the critical path. (2: 1 RH + 1 PNT)

3. **Crown axioms (PNT bureaucracy):** Unconditional, provable from Mathlib PNTA. (2)
4. **Off-path axioms:** Don't affect the crown. (~ 86)
5. **Theorems:** Machine-verified from Mathlib.
6. **Archived axioms:** Previously active, now proved. (26+)

This gradient makes progress visible. Each axiom elimination moves a statement from category 2/3 to category 5. The reduction from 56 crown axioms to 2 substantive axioms is a deleveraging of the proof's balance sheet.

4 Mathlib APIs That Worked Well

4.1 Interval Integrals

Mathlib's `MeasureTheory.integral` and `intervalIntegral` work well for L^2 norms on $(0, 1)$. Key lemma: `integral_nonneg` with `sq_nonneg` for $d_N^2 \geq 0$.

4.2 Matrix API

Mathlib's matrix library is excellent for Gram matrices. `Matrix.PosDef.det_pos` connects positive definiteness to invertibility. `Matrix.nonsing_inv` provides the inverse.

4.3 Spectral Theory

For eigenvalue bounds, use the Rayleigh quotient approach via `Matrix.IsHermitian` (which handles the real case via `starRingEnd`).

4.4 Convergence

When possible, avoid `Filter.Tendsto` and `Filter.atTop`. Algebraic bounds with explicit inequalities are easier to formalize.

5 Mathlib APIs That Required Workarounds

5.1 Infinite Series

`tsum` requires `Summable` hypotheses difficult to establish for Dirichlet series. We used truncated `Finset.sum` with explicit error bounds.

5.2 Fractional Part

`Int.fract` exists but its API is sparse for analytic number theory (integrability, measurability). Expect to prove many basic properties yourself.

5.3 Measure Theory Coercions

Moving between Bochner and Henstock–Kurzweil integrals requires care via `intervalIntegrable_iff_integra`

6 Mathlib Gaps: Two Remaining, Three Graduated

Of the five original Mathlib gaps that forced the Cathedral to introduce axioms, **three** were resolved by formalizing the required mathematics directly:

Gap	What's Missing	Difficulty	Status
Analytic continuation	Mellin integral extension	Medium	Open (2 axioms)
Mertens / PNT	$M(x) = \sum_{n \leq x} \mu(n)$ asymptotics	Medium	Partially graduated (2 axioms remain)
Montgomery–Vaughan	Dirichlet polynomial MVT	Hard	Graduated
Contour integrals	Vertical line integration	Hard	Graduated
Dedekind sums	Off-diagonal Vasyunin	Medium	Graduated

Graduated gaps. The Montgomery–Vaughan MVT was formalized as the *Gallagher MVT* via Fejér kernel (`Analysis/GallagherMVT.lean`, 0 axioms, 0 sorry). The Perron contour chain was built from scratch without Mathlib’s contour integral API (16 files, 0 axioms, 0 sorry). The off-diagonal Vasyunin integral was proved via the Cotangent Tower: piecewise partition, telescope sums, and GCD reduction (25 files, 0 axioms, 0 sorry).

Remaining gaps. The Mertens/PNT gap was substantially reduced on May 31, 2026 when 10 PNT axioms were graduated to theorems using the `PrimeNumberTheoremAnd` bridge. Two PNT bureaucracy axioms remain (`frac_error_isLittle0`, `pnt_mu_log_sq_div_k`). These would be eliminated if Mathlib exported the specific asymptotic forms of the Prime Number Theorem. The analytic continuation gap (2 axioms, **safely quarantined off the crown path**) encodes the Mellin–Fourier interchange and spectral bounds.

7 Twelve Essential Mathlib Lemmas

#	Lemma	Use
1	<code>integral_nonneg</code>	$\int f \geq 0$ when $f \geq 0$
2	<code>integral_mono_of_nonneg</code>	Bounding integrals
3	<code>sq_nonneg</code>	$x^2 \geq 0$
4	<code>Matrix.PosDef.det_pos</code>	Gram invertibility
5	<code>Matrix.nonsing_inv</code>	Matrix inversion
6	<code>Real.log_le_sub_one_of_le</code>	Log inequalities
7	<code>Real.rpow_natCast</code>	Power coercion
8	<code>Finset.sum_le_sum</code>	Bounding finite sums
9	<code>riemannZeta_ne_zero_of_one_le_re</code>	$\zeta(s) \neq 0$
10	<code>integral_univ_inv_one_add_sq</code>	$\int (1+t^2)^{-1} = \pi$
11	<code>MonotoneBounded.tendsto</code>	Monotone convergence
12	<code>Nat.Coprime.gcd_eq_one</code>	GCD properties

8 Project Structure

8.1 The Layered Module Pattern

1. **Layer 0:** Definitions. No theorems, no axioms.
2. **Layer 1:** Independent mathematical foundations.
3. **Layer 2:** Domain-specific modules (no cross-deps).

4. **Layer 3:** Bridges connecting Layer 2 modules.
5. **Layer 4:** Assembly (crown theorem).

8.2 The Axiom Registry Pattern

Maintain `Axioms.lean` documenting every axiom. Run `#print axioms` on the crown theorem after every structural change. Discrepancies are bugs.

8.3 Sorry Hygiene

1. Never `sorry` on the critical path.
2. Document every `sorry` with a comment.
3. Track sorry count as a project metric.

The Cathedral maintains: 0 sorry on the crown path.

9 Tactic Patterns

- **linarith-norm_num:** For numerical inequalities.
- **ring-then-bound:** Algebra then analysis.
- **convert-congr:** Escape hatch for type unification. Use as last resort.

10 Deprecated API Traps

Deprecated	Replacement	Status
<code>isHermitian_iff_isSymmetric</code>	(none yet)	Kept
<code>Integrable.bdd_mul'</code>	<code>Integrable.bdd_mul</code>	Updated
<code>measurable_of_continuousOn</code>	<code>Continuous.measurable</code>	Updated
<code>push_neg</code>	<code>push_not</code>	Updated (Lean 4 nightly)

When Lean warns “deprecated,” check whether the replacement exists in your Mathlib version.

11 Verification Checklist

- ☐ `lake build` passes with zero errors.
- ☐ `#print axioms crown_theorem` shows only expected axioms.
- ☐ No `sorry` on the crown path.
- ☐ All sorry markers documented.
- ☐ Ghost axiom detection passes.
- ☐ Numerical validation agrees with formal claims.
- ☐ All deprecated API calls documented.

12 Conclusion

Formalizing analytic number theory in Lean 4 is feasible today, but requires significant infrastructure work. Of five original Mathlib gaps, three were resolved by direct formalization (Gallagher MVT, Perron contour chain, off-diagonal Vasyunin). The Mertens/PNT gap was substantially reduced by the PrimeNumberTheoremAnd bridge (10 axioms graduated); 2 PNT bureaucracy axioms remain **on the crown path**, while 2 analytic continuation axioms remain **quarantined on alternative exploratory paths**. The Cathedral took ~ 68 days and produced $\sim 120,000$ lines across 390+ active files (500+ including archived explorations).

The foundational analysis shows the proof inhabits ordinary mathematics (ZFC); the universe hierarchy adds metatheoretic strength (countably many inaccessible cardinals, §2) but is never exercised by the proof content. Every assumption is named, classified, and tracked. The trust chain is auditable.

A proof is not just a certificate of truth. It is a map of trust, showing exactly where certainty ends and faith begins.

Data and Code Availability. The complete Lean 4 formalization, Rust experiments, and GPU pipeline code are available at <https://github.com/jrgochan/prime>. The repository compiles with `lake build` (Lean 4 / Mathlib, zero errors, 8,736 build jobs, zero non-sorry warnings).

References

- [1] L. de Moura et al., *The Lean 4 Theorem Prover*, CADE-28, 2021.
- [2] The Mathlib Community, *Mathlib4*, <https://github.com/leanprover-community/mathlib4>.
- [3] B. Werner, *Sets in Types, Types in Sets*, Proc. TACS 1997.
- [4] M. Carneiro, *The Type Theory of Lean*, MSc thesis, CMU, 2019.
- [5] N.G. de Bruijn, *The Mathematical Language AUTOMATH*, Springer LNCS 125 (1970).
- [6] J.R. Gochanour, *The Cathedral: A Machine-Verified Reduction of the Riemann Hypothesis*, 2026.